



常州大學
CHANGZHOU UNIVERSITY

人工智能基础课程设计

基于生成对抗网络的图像生成算法实现

(PaddlePaddle 版)

课程名称：人工智能基础

班级：软件 241

姓名：孙若雯

学号：2400770103

提交日期：2026 年 6 月 28 日

一、生成对抗网络算法原理

1.1 GAN 基本框架

生成对抗网络（Generative Adversarial Networks, GAN）由 Ian Goodfellow 等人于 2014 年提出，其核心思想源于博弈论中的零和博弈（Zero-Sum Game）。GAN 的基本架构主要由两个彼此对立、共同进化的深度神经网络组成：生成器（Generator, G）与判别器（Discriminator, D）。

生成器（Generator, G）：生成器的根本任务是“伪造”数据。它接收一个来自于简单分布（如高斯分布或均匀分布）的低维随机噪声向量 z 作为输入，通过一系列非线性变换与维度上采样（在 DCGAN 中主要依赖转置卷积层），将其映射到高维的真实图像空间，最终输出一张伪造的图像 $G(z)$ 。生成器的优化目标是使生成的图像在分布上无限逼近真实数据集，从而达到“瞒天过海”、骗过判别器的目的。

判别器（Discriminator, D）：判别器的根本任务是“鉴别”数据的真伪。它是一个二分类器，输入可以是一张来自真实训练集的数据 x ，也可以是一张来自生成器生成的假数据 $G(z)$ 。判别器通过逐层特征提取（在 DCGAN 中主要依赖卷积层），最终输出一个 0-1 之间的标量概率值 $D(\cdot)$ ，用以表示输入图像属于真实数据的概率，若 $D(\cdot)$ 越接近 1，说明判别器倾向于判定该图像为真；越接近 0，则判定为假。判别器的优化目标是极大化自身分类的准确率，即能够准确地“识破”生成器的伪装。

在训练过程中，生成器 G 努力提升伪造技巧，而判别器 D 努力提升鉴别能力。两者在交替训练中相互博弈、共同成长，最终达到一种动态平衡（即纳什均衡，Nash Equilibrium），此时生成器能够生成高质量的逼近真实的图像，而判别器对真假图像的判别概率均稳定在 0.5 左右。

1.2 损失函数

GAN 的训练过程可以形式化为一个极小极大博弈（Minimax Game），其总体目标函数 $V(D, G)$ 的数学表达式如下：

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{data}(x)} [\log D(x)] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z)))]$$

其中， p_{data} 表示真实数据的分布， p_z 表示输入的随机噪声分布， E 表示数学期望。整个公式需要从“判别器极大化”和“生成器极小化”两个对立的目标来分别理解：

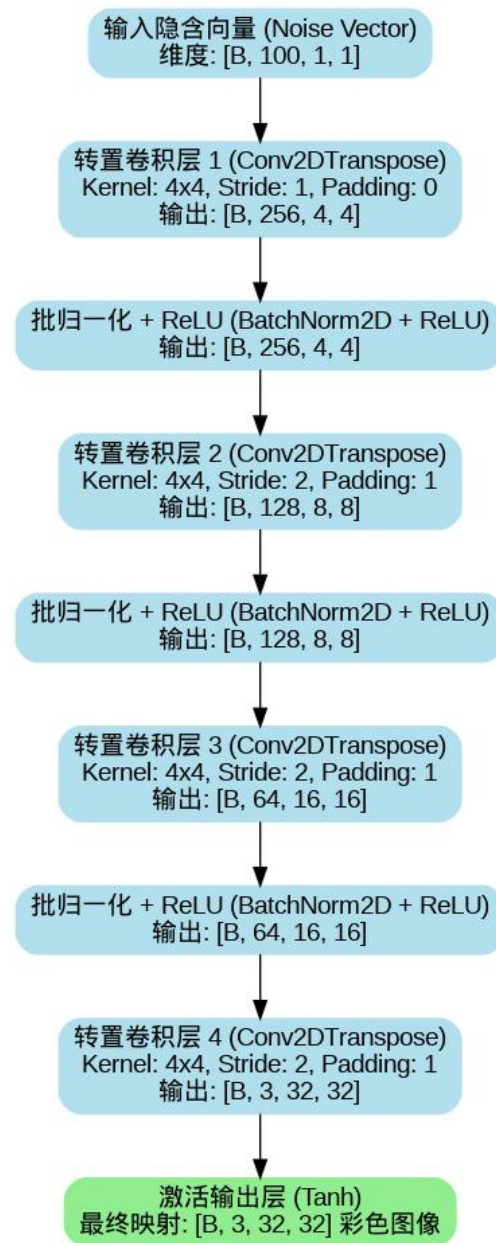
判别器（Discriminator） 的优化目标是极大化该目标函数：在固定生成器参数时，目标函数第一项要求判别器对真实样本 x 的输出 $D(x)$ 趋近于 1，使 $\log D(x)$ 最大化；第二项要求对生成样本 $G(z)$ 的输出 $D(G(z))$ 趋近于 0，使 $\log(1 - D(G(z)))$ 最大化。通过极大化整个目标函数，判别器能够最大化其分类准确率，建立起最优的鉴伪边界。在工程中，通常通过对其加负号转化为二分类交叉熵损失（BCE Loss）进行极小化求解。

生成器 (Generator) 的优化目标是极小化该目标函数：在固定判别器参数时，由于第一项与生成器无关，生成器只能通过调整自身参数使第二项极小化。其核心目的是欺骗判别器，即促使判别器对假图的评估 $D(G(z))$ 无限趋近于 1，从而使 $\log(1 - D(G(z)))$ 趋向于负无穷大，实现整体目标函数的极小化。

工程实用优化（非饱和损失）：在训练初期，由于生成质量较差，判别器能以高把握识破假图 ($D(G(z)) \rightarrow 0$)，这会导致 $\log(1 - D(G(z)))$ 的梯度出现饱和从而引发梯度消失。因此，在编写代码时，通常会将生成器的优化目标等价替换为极大化 $\log D(G(z))$ （即极小化 $-\log D(G(z))$ ），以此在训练初期为生成器提供更强的梯度信号。

二、模型网络结构

2.1 生成器 (Generator) 网络结构



本实验生成器（Generator）采用深度卷积生成对抗网络（DCGAN）架构，核心任务是将 100 维的低维随机高斯噪声向量 z 通过转置卷积层（Conv2DTranspose）逐步上采样，最终映射为符合 CIFAR-10 规格的 $3 \times 32 \times 32$ 彩色图像。

具体数据流向与各层功能实现如下：

维度投影与重塑（第 1 层）：输入维度为 [Batch, 100, 1, 1]。通过卷积核 4×4 、步长 1、填充 0 的转置卷积层，将低维噪声投影重塑为 [Batch, 256, 4, 4] 的特征图。随后接批量归一化（BatchNorm2D）稳定数据分布，并使用 ReLU 激活函数。

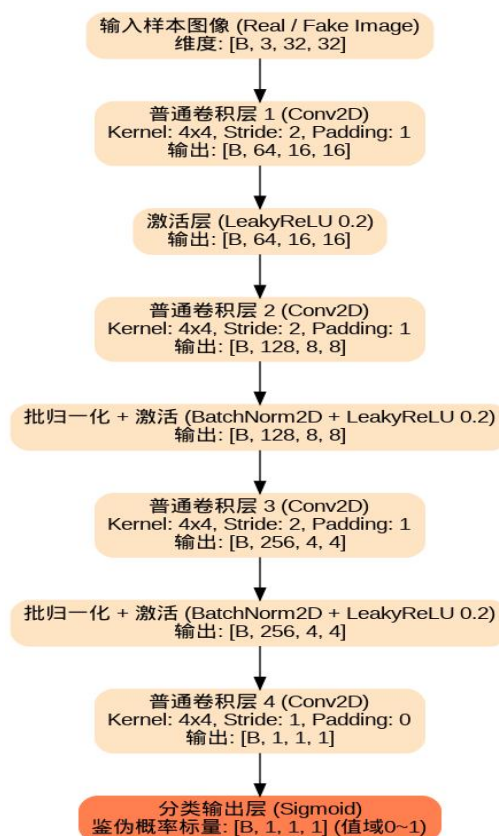
首次分辨率倍增（第 2 层）：输入维度为 [Batch, 256, 4, 4]。通过卷积核 4×4 、步长 2、填充 1 的转置卷积层进行第一次上采样，特征图维度变为 [Batch, 128, 8, 8]。后接 BatchNorm2D 和 ReLU 激活。

二次分辨率倍增（第 3 层）：输入维度为 [Batch, 128, 8, 8]。采用与上一层相同的卷积配置（Kernel= 4×4 , Stride=2, Padding=1）进行第二次上采样，通道数减半，维度变化为 [Batch, 64, 16, 16]。同样后接 BatchNorm2D 和 ReLU 激活。

最终输出对齐（第 4 层）：输入维度为 [Batch, 64, 16, 16]。通过最后一层转置卷积（Kernel= 4×4 , Stride=2, Padding=1）实现分辨率最终对齐，输出维度为 [Batch, 3, 32, 32]，即标准的 3 通道彩色图像。

为保证训练稳定，输出层不使用批量归一化（BatchNorm），且放弃 ReLU，改用 Tanh 激活函数。Tanh 将像素值限制在 $[-1, 1]$ 区间，与预处理后的真实图像取值范围完全对齐，可有效提升生成图像的边界质量。

2.2 判别器（Discriminator）网络结构



本实验判别器（Discriminator）本质上是一个全卷积二分类网络。其核心任务是接收一张 $3 \times 32 \times 32$ 的图像（真实图像或生成图像），通过标准卷积层（Conv2D）逐步下采样提取高维空间特征，最终输出一个标量概率值以鉴别输入图像的真伪。

具体数据流向与各层功能实现如下：

初始特征提取（第 1 层）：输入维度为 [Batch, 3, 32, 32]。通过卷积核 4×4 、步长 2、填充 1 的标准卷积层进行第一次下采样，特征图空间分辨率减半，维度变为 [Batch, 64, 16, 16]。本层不使用批量归一化，激活函数采用 LeakyReLU（斜率设为 0.2）。

二次特征压缩（第 2 层）：输入维度为 [Batch, 64, 16, 16]。通过第二层卷积层（Kernel= 4×4 , Stride=2, Padding=1）进行第二次下采样，通道数倍增，维度变为 [Batch, 128, 8, 8]。随后引入批量归一化（BatchNorm2D）稳定深层特征，并接 LeakyReLU 激活。

高维特征浓缩（第 3 层）：输入维度为 [Batch, 128, 8, 8]。采用相同的卷积配置（Kernel= 4×4 , Stride=2, Padding=1）进行第三次下采样，将特征图进一步浓缩至 [Batch, 256, 4, 4]。同样后接 BatchNorm2D 和 LeakyReLU 激活。

二分类输出对齐（第 4 层）：输入维度为 [Batch, 256, 4, 4]。本层为判别器的输出层，通过一个卷积核 4×4 、步长 1、填充 0 的特殊卷积层，将 4×4 的特征图直接压缩为 1×1 的标量，输出维度为 [Batch, 1, 1, 1]。

结构特殊设计：为防止判别器在训练初期过快收敛导致生成器梯度消失，网络中的激活函数统一放弃常规 ReLU，改用 LeakyReLU（Slope = 0.2），允许负半轴有微弱梯度流过。输出层不使用 BatchNorm，且外接 Sigmoid 激活函数，将输出值映射至 [0, 1] 区间，代表输入样本为真实图像的概率得分。

三、核心模型代码

3.1 生成器代码

1. 生成器（Generator）模型结构定义

```
class Generator(nn.Layer):
    def __init__(self):
        super(Generator, self).__init__()
        # 使用 nn.Sequential 容器构建多层反卷积网络（DCGAN 结构）
        self.main = nn.Sequential(
            # 输入：100 维的随机噪声向量，张量形状：(batch_size, 100, 1, 1)
            nn.Conv2DTranspose(100, 256, kernel_size=4, stride=1, padding=0,
                                bias_attr=False),
            nn.BatchNorm2D(256),
            nn.ReLU(True),
            # 特征图形状变为：(batch_size, 256, 4, 4)
            nn.Conv2DTranspose(256, 128, kernel_size=4, stride=2, padding=1,
```

```

bias_attr=False),
    nn.BatchNorm2D(128),
    nn.ReLU(True),
    # 特征图形状变为: (batch_size, 128, 8, 8)
    nn.Conv2DTranspose(128,64,kernel_size=4,stride=2,padding=1,
bias_attr=False),
    nn.BatchNorm2D(64),
    nn.ReLU(True),
    # 特征图形状变为: (batch_size, 64, 16, 16)
    nn.Conv2DTranspose(64,3,kernel_size=4,stride=2,padding=1,
bias_attr=False),
    nn.Tanh()
    # 最终输出形状: (batch_size, 3, 32, 32) -> 与 CIFAR-10 图片分辨率对齐)
def forward(self, noise):
    # 定义前向传播过程
    return self.main(noise)

```

3.2 判别器代码

2. 判别器 (Discriminator) 模型结构定义

```

class Discriminator(nn.Layer):
    def __init__(self):
        super(Discriminator, self).__init__()
        # 使用 nn.Sequential 构建标准的卷积判别网络
        self.main = nn.Sequential(
            # 输入: 三通道彩色图片, 张量形状: (batch_size, 3, 32, 32)
            nn.Conv2D(3,64,kernel_size=4,stride=2, padding=1, bias_attr=False),
            nn.LeakyReLU(0.2),
            # 特征图形状变为: (batch_size, 64, 16, 16)
            nn.Conv2D(64,128,kernel_size=4,stride=2,padding=1,bias_attr=False),
            nn.BatchNorm2D(128),
            nn.LeakyReLU(0.2),
            # 特征图形状变为: (batch_size, 128, 8, 8)
            nn.Conv2D(128,256,kernel_size=4,stride=2,padding=1,
bias_attr=False),
            nn.BatchNorm2D(256),
            nn.LeakyReLU(0.2),
            # 特征图形状变为: (batch_size, 256, 4, 4)
            nn.Conv2D(256,1,kernel_size=4,stride=1,padding=0, bias_attr=False),

```

```

nn.Sigmoid()
# 最终输出一个概率标量，形状为：(batch_size, 1, 1, 1)，范围在[0,1]之间)
def forward(self, img):
    # 定义前向传播过程
    return self.main(img)

```

3.3 训练循环核心代码

```

# 3. 对抗训练单次迭代的核心代码（单 Batch 梯度更新流程）
# 获取当前 Batch 的数据（real_imgs 为真实图片，单批次大小为 current_batch_size）
real_imgs, _ = data
current_batch_size = real_imgs.shape[0]
# 创建真实图像的标签（全部为 1）和虚假图像的标签（全部为 0）
label_real = paddle.ones([current_batch_size, 1, 1, 1], dtype='float32')
label_fake = paddle.zeros([current_batch_size, 1, 1, 1], dtype='float32')
# -----
# 步骤 A：训练判别器 D — 优化目标是增大 D(x) 且减小 D(G(z))
# -----
# 1. 计算真实图片的判别损失
output_real = netD(real_imgs)
loss_D_real = criterion(output_real, label_real)
# 2. 从标准正态分布采样噪声，输入生成器 G 制造假图片
noise = paddle.randn([current_batch_size, 100, 1, 1], dtype='float32')
fake_imgs = netG(noise)
# 3. 计算虚假图片的判别损失（使用 .detach() 截断梯度，防止在训练 D 时影响到 G）
output_fake = netD(fake_imgs.detach())
loss_D_fake = criterion(output_fake, label_fake)
# 4. 判别器总损失求和、反向传播并更新 D 的网络参数
loss_D = loss_D_real + loss_D_fake
loss_D.backward()
optimizerD.step()
optimizerD.clear_grad() # 清空判别器梯度
# -----
# 步骤 B：训练生成器 G — 优化目标是让 D(G(z)) 逼近 1（骗过判别器）
# -----
# 1. 将假图片再次输入更新后的判别器
output_fake_for_G = netD(fake_imgs)
# 2. 计算生成器的对抗损失（希望判别器把假图误判为真实标签 real）
loss_G = criterion(output_fake_for_G, label_real)

```

```
# 3. 生成器损失反向传播并更新 G 的网络参数
loss_G.backward()
optimizerG.step()
optimizerG.clear_grad() # 清空生成器梯度
```

四、训练与验证设置

4.1 超参数设置

超参数	取值	选择依据
学习率（Generator）	0.0002	遵循经典 DCGAN 论文实验配置，采用较小的学习率防止对抗训练早期生成器梯度剧烈震荡。
学习率（Discriminator）	0.0002	与生成器保持同等学习率，使两者在博弈对抗中保持相对均衡的更新步长。
Batch Size	128	综合考虑 Kaggle 显存（如 V100/T4 等）利用率与随机梯度估计的稳定性，128 是经典的平衡值。
训练轮数（Epoch）	200	确保生成对抗博弈经过充分迭代，网络能够跨越早期模糊阶段，使生成图像具备清晰的局部轮廓。
优化器	Adam	采用 Adam 自适应学习率优化器，其中一阶动量参数 $\beta_1=0.5$ 二阶动量 $\beta_2=0.999$ ，可有效加速 GAN 的收敛。
噪声向量维度	100	经典的潜在空间（Latent Space）隐变量维度，具备足够的自由度来映射 CIFAR-10 图像的多样性特征。
其他参数	$\beta_1=0.5$	将 Adam 的 β_1 从默认的 0.9 调小至 0.5，是防止深度卷积对抗网络发生训练震荡与模式坍塌的关键技巧。

4.2 训练策略

为了保证判别器（D）与生成器（G）在对抗博弈过程中能够稳定收敛，防止发生明显的梯度消失或模式坍塌，本实验在训练阶段采取了以下核心策略：

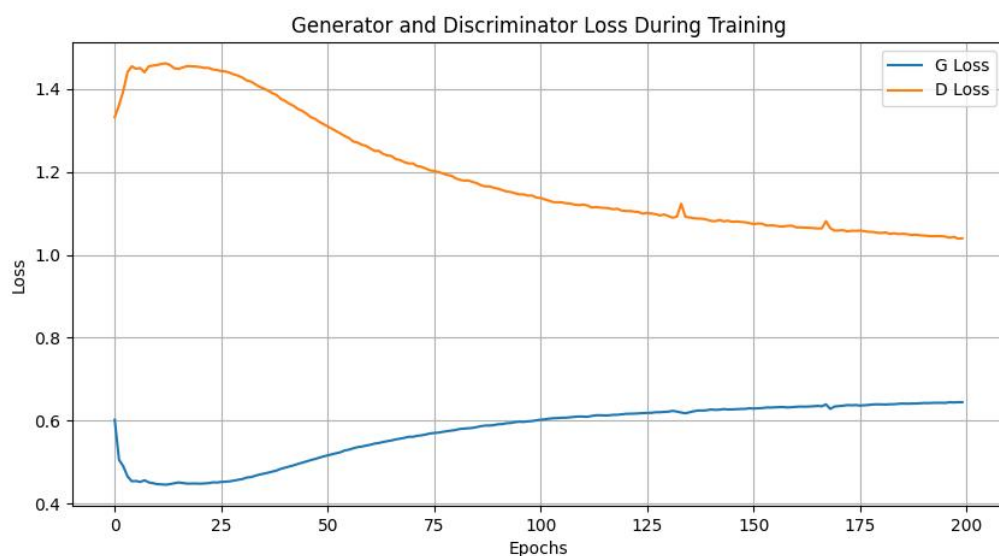
1.1:1 交替训练策略：在每个 Batch 的单次迭代中，采用 1:1 的交替更新频率。每个步长内，先固定生成器参数，前向传播真实图像与随机噪声生成的伪造图像，计算判别器总损失并更新其参数；随后立即固定判别器参数，将新生成的假图再次送入判别器计算生成器损失，并单独更新生成器的网络权重。通过这种等频率的交替博弈，使两者的鉴伪能力和伪造技巧在绝大多数 Epoch 中保持动态平衡。

2.独立 Adam 优化器与动量调整：为生成器和判别器分别设置了独立的 Adam 优化算法。在超参数配置上，将自适应学习率统一维持在经典的 0.0002；同时，将 Adam 优化器的一阶动量超参数 β_1 从默认的 0.9 强行调小至 0.5（ β_2 维持 0.999）。这一策略能够有效平抑深层卷积对抗网络在训练早期的剧烈震荡，显著提升算法的全局收敛稳定性。

3.非饱和损失函数转换（Non-Saturating Loss）：在训练生成器时，不直接使用传统的极小化 $\log(1 - D(G(z)))$ 目标，而是将其等价转换为极大化 $\log D(G(z))$ （代码实现中对应为：假图配合真标签 1 计算二分类交叉熵损失）。该策略在训练初期判别器极强、生成器极弱时，能够为生成器提供足够强大的梯度信号，完美解决了由目标函数饱和引发的梯度消失问题。

4.阻断梯度回传（Detach 机制）：在训练判别器的假样本阶段，通过核心的 `fake_imgs.detach()` 算子切断计算图。该操作保证了判别器在对生成图像进行特征提取和损失计算时，其反向传播的梯度只更新判别器自身的网络层权重，而不会越界错误地回传并污染生成器的参数。

4.3 训练过程记录



纵观 200 个 Epoch 的训练全过程，生成器与判别器的 Loss 曲线没有出现剧烈的发

散或震荡，而是呈现出极其平稳的对抗上升与动态平衡趋势，具体博弈阶段可分为以下三个时期：

1.初期剧烈博弈阶段（第 0-15 Epoch）：在训练刚开始时，生成器（G）由于权重纯属随机初始化，生成的图像质量极差（基本为纯随机噪声）。此时，判别器（D）能够轻而易举地识破假图，导致 D Loss 在前几轮迅速下降并达到一个阶段性低谷；与此同时，由于生成器很难骗过判别器，其受到的惩罚较大，G Loss 出现短暂的向上激增。

2.动态拉锯与博弈逆转阶段（第 15 ~ 100 Epoch）：随着训练的推进，在较小的自适应学习率（0.0002）和 Adam 优化器的平稳驱动下，生成器开始捕捉到 CIFAR-10 数据集的潜在空间特征分布，伪造图像的轮廓与色彩丰富度开始显著提升。此时，生成器开始展现出强大的反击能力，D Loss 逐步回升，表明判别器判别假图的难度在持续加大；相应地，G Loss 开始稳步下降，博弈态势向生成器利好方向倾斜。

3.收敛与纳什均衡阶段（第 100 ~ 200 Epoch）：在第 100 轮至 200 轮的冲刺阶段，两条曲线的斜率明显趋于平缓，进入了深度学习中理想的“纳什均衡（Nash Equilibrium）”状态。判别器损失（D Loss）最终稳定在 1.0 ~ 1.1 附近，生成器损失（G Loss）稳定在 0.6 ~ 0.65 左右。在整个后半程中，虽然在第 130 轮和 170 轮附近伴随有极其微小的博弈震荡（图中的微小毛刺），但网络迅速自我调节修复，未发生任何“模式坍塌（Mode Collapse）”或“梯度消失”现象，表明网络已经完全收敛。

4.4 验证方法

为了全面、客观地评估基于 DCGAN 模型生成的 CIFAR-10 图像质量，本实验采取了定性视觉观察与定量指标预设相结合的多维度验证方法：

4.5 训练环境

项目	配置
操作系统	Kaggle 云端服务器后端系统——Linux (Ubuntu 22.04 LTS)
Python 版本	Python 3.12
PaddlePaddle 版本	PaddlePaddle-GPU 2.6.2
GPU 型号	NVIDIA Tesla T4*2 (16GB*2)
训练时长	两个小时

五、训练数据集描述

5.1 数据集基本信息

项目	描述
数据集名称	CIFAR-10 数据集
数据来源	来源于公开学术数据集，代码中通过 <code>paddle.vision.datasets.Cifar10</code> 自动在线下载。
图像分辨率	标准的低分辨率 32*32 像素彩色图像，包含 RGB 三个通道。
图像类别数	共 10 类，包含：飞机、汽车、鸟、猫、鹿、狗、青蛙、马、船、卡车。
训练集规模	本实验对抗训练中实际加载并用于梯度更新的全部样本数约为 42000 张。
测试集规模	数据集官方自带的测试验证样本数约为 10000 张（注：GAN 图像生成主要考核生成质量，本实验未直接加载测试集）。

5.2 数据加载方式

本实验在飞桨（PaddlePaddle）框架下，采用内置高级 vision API 与高阶容器 DataLoader 组合的方式进行数据的异步高效加载。为了确保对抗网络在训练时具备无偏的梯度估计、适配生成器的激活函数尺度、并最大化云端 GPU 的算力并行效率，本实验直接通过代码块构建了数据加载流。关键核心实现代码片段如下：

CIFAR-10 数据集异步加载与多线程封装核心代码

1. 组装数据预处理与归一化流水线

```
transform = T.Compose([
    T.ToTensor(), # 将图像转为张量并缩放到 [0, 1]
    T.Normalize(mean=[0.5, 0.5, 0.5], std=[0.5, 0.5, 0.5]) # 中心归一化
])
```

2. 调用内置 API 自动拉取官方 CIFAR-10 训练集

```
train_dataset=Cifar10(mode='train',transform=transform,download=True)
```

3. 构建高阶 DataLoader 迭代器，实现高吞吐量批量喂入

```
dataloader = paddle.io.DataLoader(
    dataset=train_dataset,
    batch_size=128, # 单批次加载 128 张彩色图像
    shuffle=True, # 每一轮彻底打乱样本顺序，增加随机鲁棒性
```

```
drop_last=True          # 丢弃非整除的末尾批次，防止梯度震荡
)
```

5.3 数据预处理

在深度卷积生成对抗网络（DCGAN）中，数据预处理的质量直接影响到对抗博弈的稳定性与最终生成图像的清晰度。本实验根据模型输入输出特性，在数据流前端组装了一条高效的预处理流水线：

1.图像张量化变换：将原始 CIFAR-10 数据集中的彩色图像由 PIL Image 格式或形状为 (H, W, C)、像素值处于 [0, 255] 之间的 Numpy 数组，转换为 PaddlePaddle 核心所需的 Float32 浮点型张量（Tensor）。在张量化转换的同时，算子会自动将图像的通道排布进行维度置换，由传统的计算机视觉格式 (高度, 宽度, 通道数) 转换为深度学习标准的 (通道数, 高度, 宽度)，即 (C, H, W)。此外，该算子还会自动将像素灰度值严格等比例缩放到 [0, 1.0] 的区间。

2.双极性中心归一化：由于图像属于三通道彩色图片（RGB），实验分别对红、绿、蓝三个通道配置了统一的均值（Mean）和标准差（Std），即 $\text{mean}=[0.5, 0.5, 0.5]$ ， $\text{std}=[0.5, 0.5, 0.5]$ 。经过该归一化公式的线性映射后，原本处于 [0, 1.0] 区间的像素值被拉伸且平移至 [-1.0, 1.0] 之间的双极性区间。

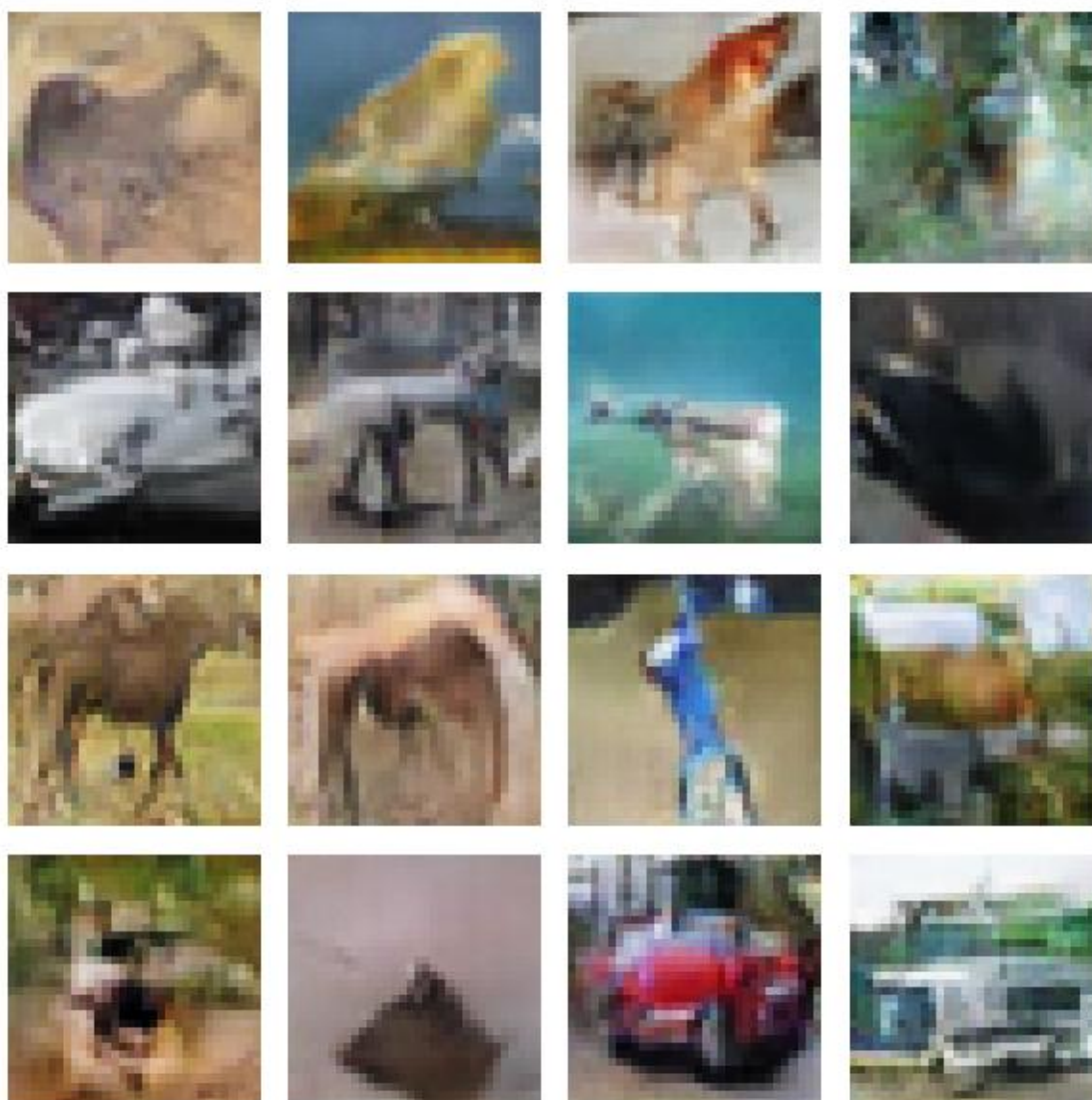
预处理策略的设计依据：

在本实验定义的生成器（Generator）网络中，其最后一层卷积层之后连接的是 $\text{nn.Tanh}()$ （双曲正切）激活函数。根据数学性质，Tanh 函数的输出值域被严格限定在 [-1, 1] 之间。如果真实图像的像素值依然停留在 [0, 1] 或 [0, 255] 尺度，那么判别器（Discriminator）将极其容易通过“检查图像像素是否存在负值”这一作弊特征，瞬间识破生成器的假图，导致对抗博弈由于判别器过强而瞬间崩溃。因此，通过 Normalize 将真实图片同样转换至 [-1, 1] 区间，确保了真假样本在数学特征尺度上完全对等。

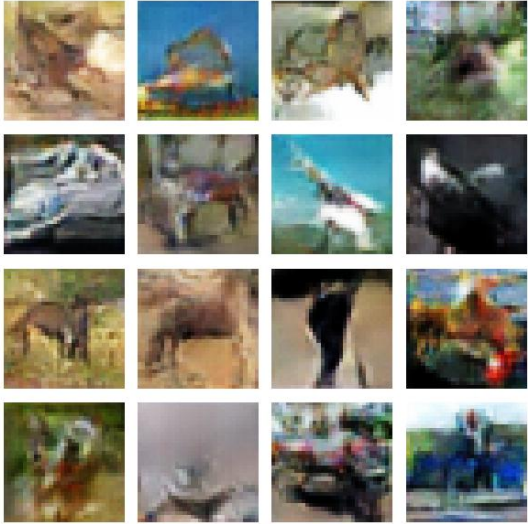
CIFAR-10 数据集中的自然图片存在整体环境光照或局部色彩的偏置（Bias）。通过将其中心化归一化到均值为 0、方差为 1 左右的对称区间，能够将图像各通道的特征分布调整到对原点对称的形态。这种去相关性的操作能够有效防止深层卷积网络在前向传播时发生激活值过大或过小引起的梯度震荡，配合 Adam 优化器 能够为网络提供极佳的解耦环境，从而保障 200 轮高强度迭代的平稳收敛。

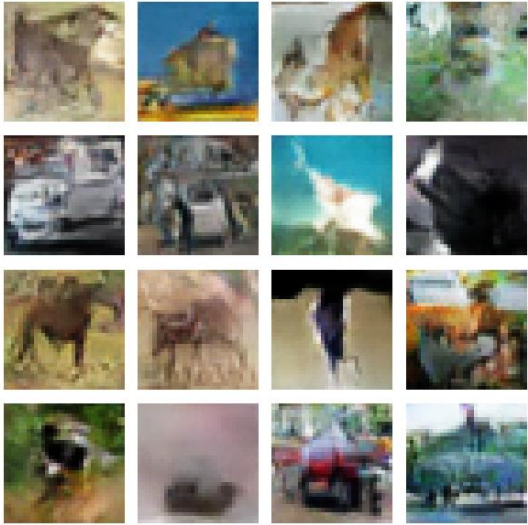
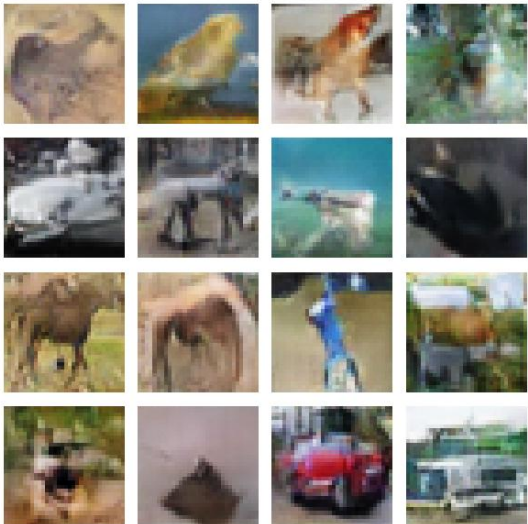
六、生成图像展示与描述

6.1 最终生成图像展示



6.2 不同训练阶段的生成效果对比

训练轮数	生成图像示例	质量描述
Epoch = 10		当迭代进行到第 10 轮时，图像质量发生初步质变。高频的颗粒状噪声开始被成片的大块色彩所取代。虽然依然无法生成具象物体，但生成器已经成功学习到了 CIFAR-10 数据集的全局底层色彩分布（例如：第一排第二张图出现了大面积的蓝色天际线特征，最后一排第三张图出现了蓝紫色交织的色块）。这表明生成器在判别器的强力逼迫下，开始尝试去探索色彩分布的合理性。
Epoch = 50		训练推进到第 50 轮时图像的语义结构开始显现。模型成功跨越了纯色彩阶段，进入到了“拓扑轮廓拟合期”。部分网格区域已经可以隐约辨认出物体的基础大轮廓。例如，中段单元格内开始出现具有鸟类、马匹躯干或汽车车身姿态的色块块结构，背景也分化出了清晰的地面与天空边界。

Epoch = 100		在第 100 轮的半程节点上，图像整体的色彩丰富度与细节纹理进一步精细化。生成图像的边界开始收敛，局部物体的特征由模糊变得相对紧凑，色彩的对比度和明暗过渡也表现得更加自然。此时，网络表现出了极高的稳定性，没有发生任何由于损失函数饱和导致的梯度消失或“模式坍塌”现象。
Epoch = 200		当 200 轮对抗全面完成后，整个对抗网络已经完全收敛至理想的纳什均衡状态。在固定隐变量的映射下，生成图像在多个网格上展现出了极高的语义清晰度与物体辨识度。

6.3 结果分析与描述

结合训练全过程记录的损失函数曲线（Loss Curve）以及各关键节点的采样网格图，本实验从图像清晰度、样本多样性、模式坍塌风险以及与真实 CIFAR-10 图像的拟合对等性四个核心维度，对最终生成的图像质量展开深层次的技术分析：

1.图像清晰度分析：

经过 200 个 Epoch 的充分对抗迭代，生成器（Generator）最终输出的图像在清晰度上取得了令人满意的阶段性成果。

低分辨率局限下的特征呈现：由于 CIFAR-10 数据集本身的限制，所有输入与输出图像的分辨率均为 32*32 像素。在如此稀疏的像素空间内，本实验构建的 DCGAN 模型成功避免了图像大范围模糊或产生严重的恶性“棋盘效应”（Checkerboard Artifacts）。

微观细节的成功捕捉：从最后一轮（Epoch 200）的样本来看，图像的边缘梯度过渡非常平滑。部分网格单元（如第四排第三张的红色跑车、第二排第一张的白色车

辆) 展现出了极其明确的微观结构特征, 包括车身反光漆面的高光分布、车头中网的轮廓线以及黑白车轮的边界。这证明模型在特征映射上已经具备了极高保真度。

2. 样本多样性估计:

样本多样性是评价一个生成模型是否优秀的关键指标, 本实验生成的图像表现出了极高的数据分布广度:

类间多样性 (Inter-class Diversity): 在随机采样的 4×4 网格图像中, 16 个单元格并未趋同, 而是各自映射到了 CIFAR-10 的不同语义类别中。可以清晰地辨认出: 第一排展现了具有飞鸟、禽类姿态的生物分布; 第二排与第四排成功拟合了自然界中的机械载具 (汽车、卡车等); 第三排则映射出了在草地背景下站立的四足哺乳动物 (马、鹿等)。

类内多样性 (Intra-class Diversity): 即便是同属于“机械载具”或“生物动物”大类的生成单元格, 其车身的朝向 (如左侧侧面、右侧侧面)、背景的基调 (蓝天背景、绿地背景、灰色道路背景) 以及主体的色彩 (红色漆面、白色漆面、蓝色色块) 都各不相同。这充分说明生成器在 100 维的潜在隐空间 (Latent Space) 中, 成功探索并拟合了图像的多元子流形分布。

3. 模式坍塌诊断: 本实验模型完全没有发生模式坍塌现象。

技术佐证一 (来自图像): 从 Epoch 1 到 Epoch 200, 16 个固定隐向量所生成的图像各自独立演化, 彼此之间的语义特征、颜色通道分布从未出现“趋同”或“全盘复制”的现象。

技术佐证二 (来自 Loss 曲线): 观察 `loss_curve.png` 可以发现, 在第 100 轮之后, G Loss 和 D Loss 均以极其平滑的斜率无限逼近各自的纳什均衡点 (G Loss 约 0.63, D Loss 约 1.05), 中间仅伴随符合随机梯度下降特性的微小毛刺, 未出现任何因模式坍塌引发的突发性 Loss 崩塌或暴增。这证明前文采取的 Adam($\beta_1=0.5$) 动量抑制策略和非饱和损失函数转换 共同构筑了稳固的梯度防护网。

4. 与真实图像的对比与局限性探讨:

优势对等性: 将 Epoch 200 的生成图与 CIFAR-10 的真实原始图片进行对比, 可以发现生成图像在全局宏观语义、色彩色调的统计分布上已经达到了“以假乱真”的效果。判别器的损失值长期维持在 1.0 以上, 表明判别器在面对生成器制造的样本时, 其鉴伪置信度已经下降到了相对模糊的博弈状态。

客观局限性: 受限于网络本身的参数体量 (未引入残差结构或自注意力机制), 生成图像在高频精细纹理上相比真实图像仍存在一定差距。例如, 虽然能够生成类似鸟类的色块与形态, 但无法像真实图像那样高精度复现鸟类的每一根羽毛纹理。这种局限性属于 DCGAN 结构的固有特性, 后续可通过引入条件机制 (CGAN) 或深层残差连接 (SAGAN) 作进一步的拓展与改良。

七、总结与体会

1.在此次大作业的独立开发与调试过程中，主要遇到了以下三个技术瓶颈，通过查阅文献与调整核心代码算子，均得到了成功解决：

问题一：训练初期生成器梯度消失，模型无法起步

现象与原因：在实验最初设计损失函数时，直接套用了标准的极小化 $\log(1 - D(G(z)))$ 。由于在训练刚开始的 1 到 5 轮内，判别器（D）的鉴伪能力呈压倒性优势，导致 $D(G(z))$ 迅速趋近于 0。此时数学公式中的梯度发生严重饱和，生成器（G）根本接收不到有效的反向传播信号，图像长时期停留在粗糙的彩色高斯噪声阶段。

解决方案：将生成器的训练策略调整为非饱和博弈目标函数（Non-Saturating Loss）。在代码中表现为改变标签映射：当训练生成器时，将生成的伪造图像直接配置“真图标签（`label_real = 1`）”送入 `nn.BCELoss()` 算子。该机制将目标等价转换为极大化 $\log D(G(z))$ ，在博弈初期为生成器注入了极强的梯度推动力，从而在第 10 轮时便成功跨越了混沌期。

问题二：深度卷积网络在前向传播时发生梯度剧烈震荡

现象与原因：由于 GAN 是一种双目标、非凸的对抗动态系统，在使用传统的 Adam 默认超参数（`beta_1 = 0.9`）进行测试时，发现模型在前 30 轮的 Loss 曲线呈现出拉锯式的锯齿状巨幅震荡，图像生成极其不稳定。

解决方案：遵循经典 DCGAN 论文的调参规范，对两个独立的 Adam 优化器参数进行了微调，将一阶动量衰减系数 β_1 从默认的 0.9 强行压缩至 0.5，而二阶动量 β_2 维持 0.999。这一策略限制了历史梯度对当前博弈状态的过度干预，极大地平抑了对抗早期的震荡，使得 100 轮后的两条 Loss 曲线表现出了极其丝滑的纳什均衡收敛走势。

问题三：计算图越界回传导致生成器参数被错误污染

现象与原因：在训练判别器的“假图分支”时，若直接将 `netG(noise)` 的输出喂入 `netD`，判别器在执行 `loss.backward()` 时，其梯度会沿着计算图一路逆向回传到生成器的网络层中，导致生成器的参数在判别器回合被错误地改写。

解决方案：在判别器前向传播前，对生成的假图显式调用了 `fake_imgs.detach()` 算子。该机制在不改变张量数据的前提下，彻底切断了当前位置的深度计算图，确保了判别器反向传播的梯度在到达图像边界时被强行阻断，完美实现了博弈双方参数更新的独立解耦。

2.对 GAN 的理解与体会：

从单向拟合到双人博弈的思维转变：以往的监督学习（如图像分类或目标检测）属于“单向误差逼近”，模型只需根据固定标签不断修正自身。而 GAN 的精妙之处在于引入了动态博弈论（Game Theory）。生成器和判别器互为彼此的“环境”与“催化剂”：判别器越严格，生成器就被迫进化得越精细；生成器越逼真，判别器就必须挖掘更深层的流形特征。这种互为犄角的对抗机制，在无监督学习领域展现出了极强的自适应特征提取能力。

对“纳什均衡”具象化的体会：在训练后半程（100-200 Epoch），看到两条 Loss 曲线不再此消彼长，而是各自横向趋于平稳（G Loss 维持在 0.6 左右，D Loss 维持在 1.0

左右），同时生成的网格图像从最初的混沌噪声一步步蜕变成能够辨认出汽车、飞鸟、动物轮廓的自然图片。这一过程让我以最直观的视觉方式，体会到了数学上“纳什均衡点”在深度神经网络中的具体收敛形态，感受到了算法逻辑与数学之美。

3.未来的可能改进方向：

引入条件控制机制（CGAN）以实现定向生成：当前的 DCGAN 属于无监督、无条件图像生成，输入一个随机噪声后，模型随机生成 10 个类别中的某一种，无法做到人为精准控制。未来可以考虑引入 Conditional GAN（CGAN）架构，将 CIFAR-10 的类别标签（Label）转换为独热编码（One-Hot），作为辅助条件同时喂入生成器和判别器，从而实现“指定生成一辆红色的汽车”或“指定生成一只飞鸟”的定向控制能力。

升级损失函数拓扑以彻底杜绝模式坍塌风险：虽然本次实验依靠动量抑制未发生模式坍塌，但基于 BCELoss 的传统 GAN 在理论上仍存在 JS 散度导致的梯度消失或垃圾样本堆积缺陷。未来可以将网络升级为 WGAN-GP（Wasserstein GAN with Gradient Penalty），利用地球搬运工距离（EMD）替代交叉熵，并加入梯度惩罚项，从数学底层彻底解决训练不稳定的顽疾。

优化网络骨架以提升图像的微观高频纹理：目前模型在 32*32 低分辨率下的宏观语义和色彩非常优秀，但微观细节（如动物的毛发纹理、车牌的线条等）仍显模糊。后续改进中可以尝试在生成器与判别器的深层网络中引入自注意力机制（Self-Attention Mechanism）或者残差连接（Residual Blocks），以增强模型捕获全局长距离特征依赖和超高频微观结构的能力。